

Chapter 5

C Functions

C How to Program, 9/e, GE

Objectives

In this chapter, you'll:

- Construct programs modularly from small pieces called functions.
- Use common math functions in the C standard library.
- Create new functions.
- Use the mechanisms that pass information between functions.
- Learn how the function call/return mechanism is supported by the function call stack and stack frames.
- Use simulation techniques based on random number generation.
- Write and use functions that call themselves.

5.1 Introduction

- Most computer programs that solve real-world problems are much **larger** than the programs presented in the first few chapters.
- Experience has shown that the best way to develop and maintain a large program is to construct it from **smaller pieces**, each of which is **more manageable** than the original program.
- This technique is called **divide and conquer**.
- This chapter describes some key features of the C language that facilitate the **design, implementation, operation** and **maintenance** of **large** programs.

5.2 Modularizing Programs in C

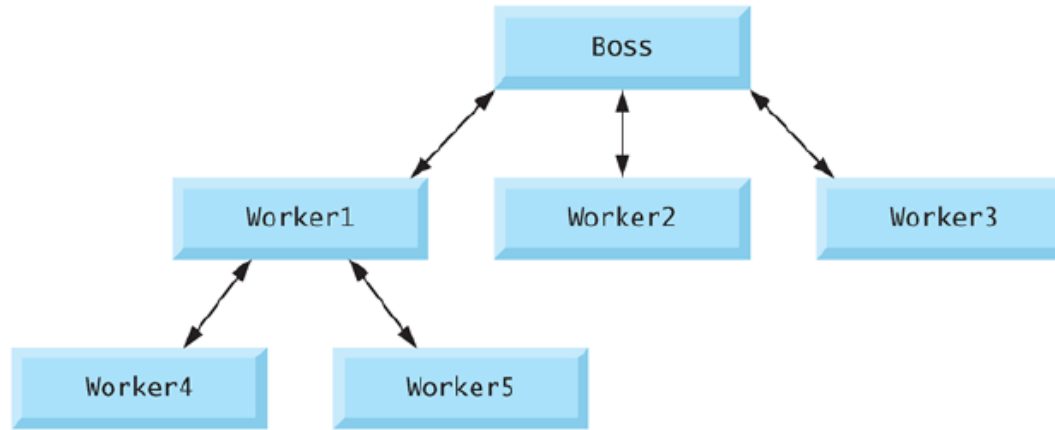
- **Functions** are used to modularize programs
- C programs are typically written by combining new functions you write with *prepackaged* functions available in the **C standard library**.
- The C standard library provides a rich collection of functions for performing common *mathematical calculations, string manipulations, character manipulations, input/output*, and many other **useful operations**.

5.2 Modularizing Programs in C (Cont.)

- The functions `printf`, `scanf` and `pow` that we've used in previous chapters are standard library functions.
- You can write your **own** functions to define tasks that may be used at many points in a program.
 - These are sometimes referred to as **programmer-defined functions**.
 - The statements defining the function are written **only once**, and the statements are **hidden** from other functions.
 - Functions are **invoked** by a **function call**, which specifies the **function name** and provides **information** (as **arguments**) that the called function needs to perform its **designated task**.

5.2 Modularizing Programs in C (Cont.)

- A boss (the **calling function** or **caller**) asks a worker (the **called function**) to perform a task and report back when the task is done.



| Hierarchical boss-function/worker-function relationship.

- A function needing to display information on the screen calls the worker function `printf` to perform that task
- `printf` displays the information and reports back—or **returns**—to the calling function when its task is completed.
- The boss function does **not** know how the worker function performs its designated tasks.

5.3 Math Library Functions

- **Math** library functions allow you to perform certain common mathematical calculations.
 - Functions are normally used in a program by writing the **name** of the function followed by a **left** parenthesis followed by the **argument** (or a comma-separated list of arguments) of the function followed by a **right** parenthesis.
 - A programmer desiring to calculate and print the square root of 900.0 , you might write

```
printf("%.2f", sqrt(900.0));
```
 - Function arguments may be **constants**, **variables**, or **expressions**.
 - If $c1 = 13.0$, $d = 3.0$ and $f = 4.0$, then the statement

```
printf("%.2f", sqrt(c1 + d * f));
```

 - calculates and prints the square root of $13.0 + 3.0 * 4.0 = 25.0$, namely 5.00 .

Function	Description	Example
<code>sqrt(x)</code>	square root of x	<code>sqrt(900.0)</code> is 30.0 <code>sqrt(9.0)</code> is 3.0
<code>cbrt(x)</code>	cube root of x (C99 and C11 only)	<code>cbrt(27.0)</code> is 3.0 <code>cbrt(-8.0)</code> is -2.0
<code>exp(x)</code>	exponential function e^x	<code>exp(1.0)</code> is 2.718282 <code>exp(2.0)</code> is 7.389056
<code>log(x)</code>	natural logarithm of x (base e)	<code>log(2.718282)</code> is 1.0 <code>log(7.389056)</code> is 2.0
<code>log10(x)</code>	logarithm of x (base 10)	<code>log10(1.0)</code> is 0.0 <code>log10(10.0)</code> is 1.0 <code>log10(100.0)</code> is 2.0
<code>fabs(x)</code>	absolute value of x as a floating-point number	<code>fabs(13.5)</code> is 13.5 <code>fabs(0.0)</code> is 0.0 <code>fabs(-13.5)</code> is 13.5
<code>ceil(x)</code>	rounds x to the smallest integer not less than x	<code>ceil(9.2)</code> is 10.0 <code>ceil(-9.8)</code> is -9.0
<code>floor(x)</code>	rounds x to the largest integer not greater than x	<code>floor(9.2)</code> is 9.0 <code>floor(-9.8)</code> is -10.0
<code>pow(x, y)</code>	x raised to power y (x^y)	<code>pow(2, 7)</code> is 128.0 <code>pow(9, .5)</code> is 3.0
<code>fmod(x, y)</code>	remainder of x/y as a floating-point number	<code>fmod(13.657, 2.333)</code> is 1.992
<code>sin(x)</code>	trigonometric sine of x (x in radians)	<code>sin(0.0)</code> is 0.0
<code>cos(x)</code>	trigonometric cosine of x (x in radians)	<code>cos(0.0)</code> is 1.0
<code>tan(x)</code>	trigonometric tangent of x (x in radians)	<code>tan(0.0)</code> is 0.0

Fig. 5.2 | Commonly used math library functions. (Part 2 of 2.)

5.4 Functions

- Functions allow you to **modularize** a program.
 - All variables defined in function definitions are **local variables**—they can be accessed *only* in the function in which they're defined.
 - Most functions have a list of **parameters** that provide the means for communicating information between functions.
 - A function's parameters are also **local variables** of that function.

5.4 Functions (Cont.)

- There are several motivations for “functionalizing” a program.
 - The **divide-and-conquer** approach makes program development more **manageable**.
 - Another motivation is **software reusability**
 - using existing functions as *building blocks* to create new programs.
 - Software **reusability** is a major factor in the *object-oriented programming* movement that you’ll learn more about when you study languages derived from C, such as C++, Java and C# (pronounced “C sharp”).
 - A third motivation is to **avoid repeating code** in a program.
- Packaging code as a function allows the code to be executed from other locations in a program simply by **calling** the function.

5.5 Function Definitions

- Each program we've presented has consisted of a function called **main** that called standard library functions to accomplish its tasks.

- The format of a function definition is

```
return-value-type function-name(parameter-list)  
{  
    definitions  
    statements  
}
```

- The *function-name* is any **valid** identifier.
- The *return-value-type* is the data type of the result returned to the caller.
- The *return-value-type* **void** indicates that a function does **not** return a value.
- Together, the *return-value-type*, *function-name* and *parameter-list* are sometimes referred to as the function **header**.
- The parameter-list is a **comma-separated** list that specifies the parameters received by the function when it's called.

```

1 // Fig. 5.3: fig05_03.c
2 // Creating and using a programmer-defined function.
3 #include <stdio.h>
4
5 int square(int y); // function prototype
6
7 int main(void)
8 {
9     // loop 10 times and calculate and output square of x each time
10    for (int x = 1; x <= 10; ++x) {
11        printf("%d ", square(x)); // function call
12    }
13    puts("");
14 }
15
16 // square function definition returns the square of its parameter
17 int square(int y) // y is a copy of the argument to the function
18 {
19     return y * y; // returns the square of y as an int
20 }
21

```

invoked or called

a copy of the value of x in the parameter y.

1 4 9 16 25 36 49 64 81 100

return-value-type function-name(parameter-list)

{

*definitions
statements*

}

5.5 Function Definitions (Cont.)

- There are **three** ways to return control from a called function to the point at which a function was invoked.
 - If the function **does not** return a result, control is returned simply when the **function-ending right brace** is reached, or by executing the statement
return;
 - If the function **does** return a result, the statement
return expression;
returns the value of *expression* to the caller.

5.5 Function Definitions (Cont.)

main's Return Type

- Notice that `main` has an `int` return type.
- The return value of `main` is used to indicate whether the program executed **correctly**.
- In earlier versions of C, we'd **explicitly** place
`return 0;`
at the end of `main`
 - `0` indicates that a program ran **successfully**.
- The C standard indicates that `main` **implicitly** returns 0 if you to **omit** the preceding statement
 - as we've done throughout this book.
- You can explicitly return **non-zero values** from `main` to indicate that a **problem** occurred during your program's execution.

5.5 Function Definitions (Cont.)

Function maximum

```
1 // Fig. 5.4: fig05_04.c
2 // Finding the maximum of three integers.
3 #include <stdio.h>
4
5 int maximum(int x, int y, int z); // function prototype
6
7 int main(void)
8 {
9     int number1; // first integer entered by the user
10    int number2; // second integer entered by the user
11    int number3; // third integer entered by the user
12
13    printf("%s", "Enter three integers: ");
14    scanf("%d%d%d", &number1, &number2, &number3);
15
16    // number1, number2 and number3 are arguments
17    // to the maximum function call
18    printf("Maximum is: %d\n", maximum(number1, number2, number3));
19 }
20
```

```
21 // Function maximum definition
22 // x, y and z are parameters
23 int maximum(int x, int y, int z)
24 {
25     int max = x; // assume x is largest
26
27     if (y > max) { // if y is larger than max,
28         max = y; // assign y to max
29     }
30
31     if (z > max) { // if z is larger than max,
32         max = z; // assign z to max
33     }
34
35     return max; // max is largest value
36 }
```

Enter three integers: 22 85 17
Maximum is: 85

Enter three integers: 47 32 14
Maximum is: 47

Enter three integers: 35 8 79
Maximum is: 79

Program Demo

- Fig. 5.3
- Fig. 5.4

5.6 Function Prototypes: A Deeper Look

- An important feature of C is the **function prototype**.
 - This feature was borrowed from C++.
- The compiler uses **function prototypes** to validate function calls.
- Early versions of C did *not* perform this kind of checking, so it was possible to call functions **improperly** without the compiler detecting the errors.
 - Such calls could result in fatal **execution-time errors** or **nonfatal errors** that caused subtle, difficult-to-detect problems.

5.6 Function Prototypes: A Deeper Look (Cont.)

- The function prototype for `maximum` in Fig. 5.4 (line 5) is

```
// function prototype
int maximum(int x, int y, int z);
```

- It states that `maximum` takes **three** arguments of type `int` and returns a result of type `int`.
- Notice that the function prototype is the **same** as the first line of `maximum`'s function definition.

5.6 Function Prototypes: A Deeper Look (Cont.)

Compilation Errors

- A function call that does **not match** the function prototype is a **compilation error**.
- An error is also generated if the function prototype and the function definition **disagree**.
- For example, in Fig. 5.4, if the function prototype had been written

```
void maximum(int x, int y, int z);
```
- the compiler would generate an error because the **void** return type in the function prototype would **differ** from the **int** return type in the function header.

5.6 Function Prototypes: A Deeper Look (Cont.)

Argument Coercion and “Usual Arithmetic Conversion Rules”

- Another important feature of function prototypes is the **coercion (強迫) of arguments**
 - The forcing of arguments to the appropriate type.
 - The math library function `sqrt` can be called with an **integer** argument even though the function prototype in `<math.h>` specifies a **double** parameter, and the function will still **work correctly**.
 - The statement

```
printf("%.3f\n", sqrt(4));
```

correctly evaluates `sqrt(4)` and prints the value `2.000`.

5.6 Function Prototypes: A Deeper Look (Cont.)

- In our `sqrt` example above, an `int` is **automatically** converted to a `double` **without** changing its value.
- However, a `double` converted to an `int` *truncates* the fractional part of the `double` value, thus changing the original value.
- Converting **large** integer types to **small** integer types (e.g., `long` to `short`) may also result in **changed** values.

5.6 Function Prototypes: A Deeper Look (Cont.)

- The usual arithmetic conversion rules for a **mixed-type** expression containing at least one floating-point value are:
 - If one of the values is a **long double**, the other is converted to a long double.
 - If one of the values is a **double**, the other is converted to a double.
 - If one of the values is a **float**, the other is converted to a float.

Data type	printf conversion specification	scanf conversion specification
<i>Floating-point types</i>		
long double	%Lf	%Lf
double	%f	%lf
float	%f	%f
<i>Integer types</i>		
unsigned long long int	%llu	%llu
long long int	%lld	%lld
unsigned long int	%lu	%lu
long int	%ld	%ld
unsigned int	%u	%u
int	%d	%d
unsigned short	%hu	%hu
short	%hd	%hd
char	%c	%c

Fig. 5.5 | Arithmetic data types and their conversion specifications.

5.6 Function Prototypes: A Deeper Look (Cont.)

- Converting values to a **lower** type normally results in an **incorrect** value.
 - Therefore, a value can be converted to a lower type only by **explicitly** assigning the value to a variable of **lower** type, or by using a **cast** operator.
- Function argument values are converted to the parameter **types** in a function prototype as if they were being **assigned** directly to variables of those types.
 - If our square function that uses an integer parameter (Fig. 5.3) is called with a floating-point argument, the argument is converted to **int** (a lower type), and square usually returns an **incorrect** value.
 - For example, `square(4.5)` returns 16, not 20.25.

```

1 // Fig. 5.3: fig05_03.c
2 // Creating and using a programmer-defined function.
3 #include <stdio.h>
4
5 int square(int y); // function prototype
6
7 int main(void)
8 {
9     // loop 10 times and calculate and output square of x each time
10    for (int x = 1; x <= 10; ++x) {
11        printf("%d ", square(x)); // function call
12    }
13
14    puts("");
15 }
16
17 // square function definition returns the square of its parameter
18 int square(int y) // y is a copy of the argument to the function
19 {
20     return y * y; // returns the square of y as an int
21 }

```

1 4 9 16 25 36 49 64 81 100

Fig. 5.3 | Creating and using a programmer-defined function.

5.7 Function Call Stack and Stack Frames

- To understand how C performs function calls, we first need to consider a data structure (i.e., collection of related data items) known as a **stack**.
 - Think of a stack as analogous to a pile of dishes.
 - When a dish is placed on the pile, it's normally placed at the **top** (referred to as **pushing** the dish onto the stack).
 - Similarly, when a dish is removed from the pile, it's normally removed from the **top** (referred to as **popping** the dish off the stack).
 - Stacks are known as **last-in, first-out (LIFO)** data structures
 - the *last* item pushed (inserted) on the stack is the *first* item popped (removed) from the stack.

5.7 Function Call Stack and Stack Frames (Cont.)

- An important mechanism for computer science students to understand is the **function call stack** (sometimes referred to as the **program execution stack**).
- This data structure—working “behind the scenes”—supports the function **call/return** mechanism.
- It also supports the **creation, maintenance** and **destruction** of each called **function's local variables**.

5.7 Function Call Stack and Stack Frames (Cont.)

- As each function is called, it may call other functions, which may call other functions
 - all before any function returns.
- Each function eventually must **return control** to the function that called it.
- So, we must keep track of the **return addresses** that each function needs to return control to the function that called it.
- The function call **stack** is the perfect data structure for handling this information.

5.7 Function Call Stack and Stack Frames (Cont.)

- Each time a function calls another function, an entry is *pushed* onto the stack.
- This entry, called a **stack frame**, contains the ***return address*** that the called function needs in order to return to the calling function.
- If the called function returns, instead of calling another function before returning, the stack frame for the function call is *popped*, and control transfers to the return address in the ***popped*** stack frame.

5.7 Function Call Stack and Stack Frames (Cont.)

- Each called function always finds the information it needs to return to its caller at the *top* of the call stack.
- And, if a function makes a call to another function, a **stack frame** for the new function call is simply pushed onto the call stack.
- Thus, the return address required by the newly called function to return to its caller is now located at the *top* of the stack.

5.7 Function Call Stack and Stack Frames (Cont.)

- The stack frames have another important responsibility.
- Most functions have *automatic variables* – parameters and some or all of their **local** variables.
 - Automatic variables need to exist while a function is executing.
 - They need to remain **active** if the function makes calls to other functions.
 - But when a called function **returns** to its caller, the called function's automatic variables need to “**go away.**”

5.7 Function Call Stack and Stack Frames (Cont.)

- The called function's **stack frame** is a perfect place to reserve the memory for *automatic variables*.
- That stack frame exists only as long as the called function is **active**.
- When that function **returns**—and no longer needs its local automatic variables—its stack frame is *popped* from the stack, and those local automatic variables are **no longer** known to the program.

5.7 Function Call Stack and Stack Frames (Cont.)

- Of course, the amount of memory in a computer is **finite**, so only **a certain amount of memory** can be used to store **stack frames** on the function call stack.
- If **more** function calls occur **than** can have their stack frames stored on the function call stack, a ***fatal error*** known as a **stack overflow** occurs.

Example for *Function Call Stack in Action*

```
1 // Fig. 5.6: fig05_06.c
2 // Demonstrating the function call stack
3 // and stack frames using a function square.
4 #include <stdio.h>
5
6 int square(int); // prototype for function square
7
8 int main()
9 {
10     int a = 10; // value to square (local automatic variable in main)
11
12     printf("%d squared: %d\n", a, square(a)); // display a squared
13 }
14
15 // returns the square of an integer
16 int square(int x) // x is a local variable
17 {
18     return x * x; // calculate square and return result
19 }
```

10 squared: 100

Fig. 5.6 | Demonstrating the function call stack and stack frames using a function square.

Function Call Stack in Action

- First the **operating system** calls `main`—this **pushes** a stack frame onto the stack.
- The stack frame tells `main` **how** to return to the operating system (i.e., transfer to return address **R1**) and contains the space for `main`'s automatic variable (i.e., **a**, which is initialized to **10**).

Step 1: Operating system invokes `main` to execute application

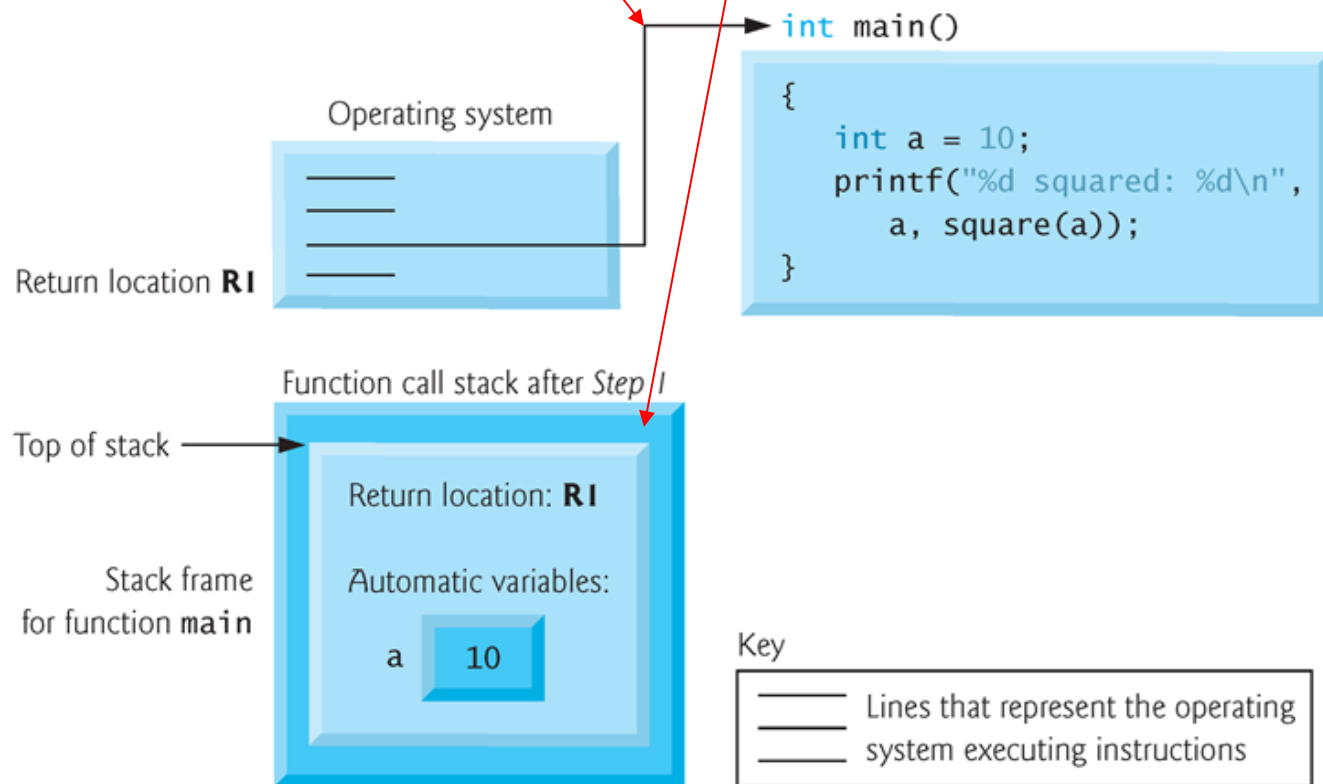


Fig. 5.7 | Function call stack after the operating system invokes `main` to execute the program.

Step 2: `main` invokes function `square` to perform calculation

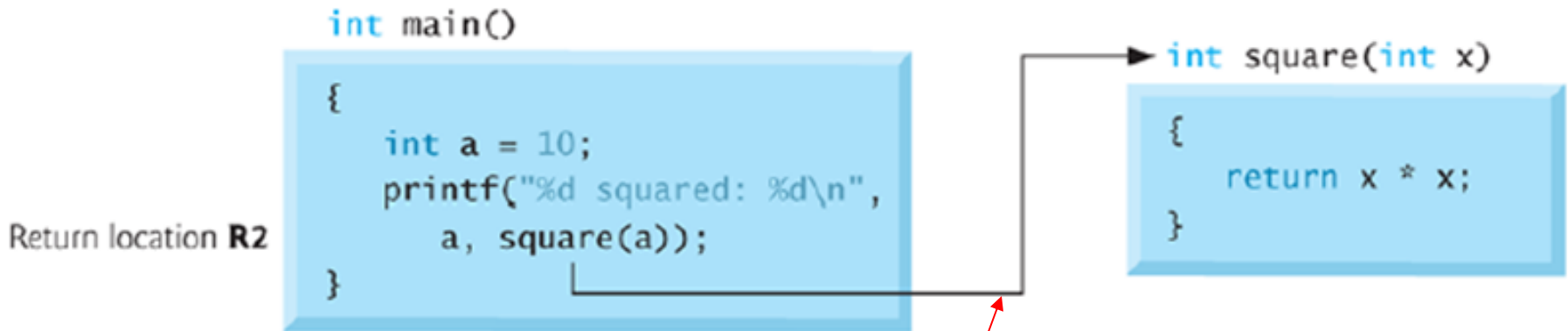
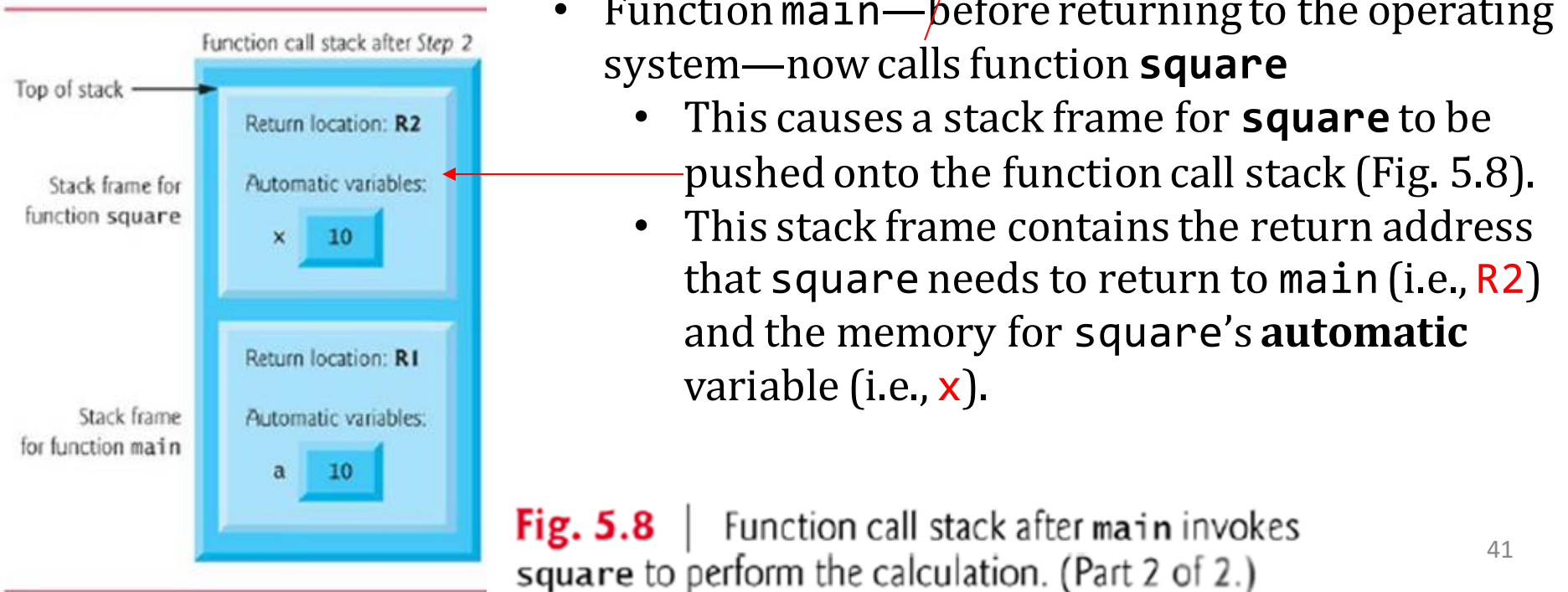


Fig. 5.8 | Function call stack after `main` invokes `square` to perform the calculation. (Part 1 of 2.)



Step 3: `square` returns its result to `main`

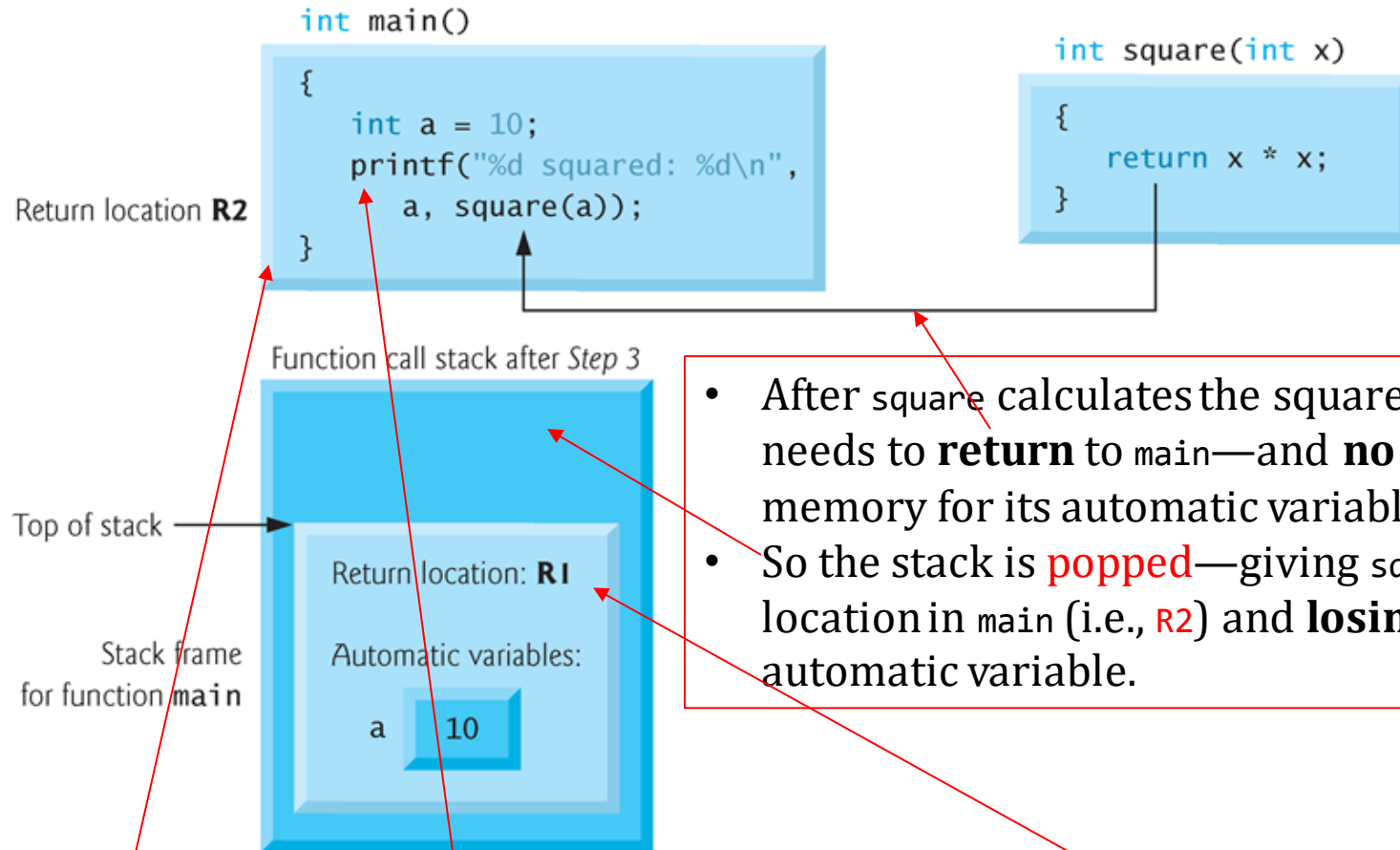


Fig. 5.9 | Function call stack after function `square` returns to `main`.

- Function `main` now displays the result of calling `square`
- Reaching the closing right brace of `main` causes its stack frame to be **popped** from the stack, gives `main` the address it needs to return to the operating system (i.e., **R1** in Fig. 5.7) and causes the memory for `main`'s automatic variable (i.e., **a**) to become **unavailable**.

Program Demo

- Fig. 5.6

5.8 Headers

- Each standard library has a corresponding **header** containing the function prototypes for all the functions in that library and **definitions** of various data types and constants needed by those functions.
- Figure 5.6 lists alphabetically some of the standard library **headers** that may be included in programs.
- The term “**macros**” that’s used several times in Fig. 5.6 is discussed in detail in Chapter 13.
- You can create **custom** headers.
- **Programmer-defined headers** should also use the **.h** filename extension.

5.8 Headers (Cont.)

- A **programmer-defined header** can be included by using the `#include` preprocessor **directive**.
- For example, if the prototype for our square function was located in the header `square.h`, we'd include that **header** in our program by using the following **directive** at the top of the program:

```
#include "square.h"
```

Header	Explanation
<code><assert.h></code>	Contains information for adding diagnostics that aid program debugging.
<code><ctype.h></code>	Contains function prototypes for functions that test characters for certain properties, and function prototypes for functions that can be used to convert lowercase letters to uppercase letters and vice versa.
<code><errno.h></code>	Defines macros that are useful for reporting error conditions.
<code><float.h></code>	Contains the floating-point size limits of the system.
<code><limits.h></code>	Contains the integral size limits of the system.
<code><locale.h></code>	Contains function prototypes and other information that enables a program to be modified for the current locale on which it's running. The notion of locale enables the computer system to handle different conventions for expressing data such as dates, times, currency amounts and large numbers throughout the world.
<code><math.h></code>	Contains function prototypes for math library functions.
<code><setjmp.h></code>	Contains function prototypes for functions that allow bypassing of the usual function call and return sequence.
<code><signal.h></code>	Contains function prototypes and macros to handle various conditions that may arise during program execution.

Fig. 5.10 | Some of the standard library headers. (Part 1 of 2.)

Header	Explanation
<code><stdarg.h></code>	Defines macros for dealing with a list of arguments to a function whose number and types are unknown.
<code><stddef.h></code>	Contains common type definitions used by C for performing calculations.
<code><stdio.h></code>	Contains function prototypes for the standard input/output library functions, and information used by them.
<code><stdlib.h></code>	Contains function prototypes for conversions of numbers to text and text to numbers, memory allocation, random numbers and other utility functions.
<code><string.h></code>	Contains function prototypes for string-processing functions.
<code><time.h></code>	Contains function prototypes and types for manipulating the time and date.

Fig. 5.10 | Some of the standard library headers. (Part 2 of 2.)

5.9 Passing Arguments By Value and By Reference

- In many programming languages, there are **two** ways to pass arguments
 - pass-by-value
 - When arguments are *passed by value*, a **copy** of the argument's value is made and passed to the called function.
 - Changes to the copy do **not** affect an original variable's value in the caller.
 - Pass-by-value should be used whenever the **called** function does **not** need to modify the value of the caller's original variable.
 - In C, all **arguments** are passed by value.
 - pass-by-reference
 - When an argument is passed by reference, the caller allows the called function to **modify** the original variable's value.
 - Pass-by-reference should be used only with trusted called functions that **need to modify** the original variable.
 - In Chapter 6, we'll see that **array** arguments are automatically passed by reference for performance reasons.

5.10 Random Number Generation

- We now take a brief and, hopefully, entertaining diversion into *simulation* and *game playing*.
- The element of chance can be introduced into computer applications by using the C standard library function `rand` from the `<stdlib.h>` header.
- Consider the following statement:
 `i = rand();`
 - The `rand` function generates an integer between **0** and **RAND_MAX** (a symbolic constant defined in the `<stdlib.h>` header).

5.10 Random Number Generation (Cont.)

- Standard C states that the value of `RAND_MAX` must be at least **32767**, which is the maximum value for a **two-byte** (i.e., 16-bit) integer.
 - The programs in this section were tested on **Microsoft Visual C++** with a maximum `RAND_MAX` value of **32767** and on GNU **gcc** and **Xcode** LLVM with a `RAND_MAX` value of **2147483647**.
 - If `rand` truly produces integers at random, every number between **0 and `RAND_MAX`** has an **equal** chance (or probability) of being chosen each time **`rand`** is called.
- The range of values produced directly by **`rand`** is often **different** from what's needed in a **specific** application.

5.10 Random Number Generation (Cont.)

- For example, a program that simulates coin tossing might require only **0** for “**heads**” and **1** for “**tails**.”
- A dice-rolling program that simulates a **six-sided** die would require random integers from **1 to 6**.

Rolling a Six-Sided Die

- To demonstrate **rand**, let’s develop a program to **simulate** 20 rolls of a six-sided die and **print** the value of each roll.
- The function prototype for function **rand** is in `<stdlib.h>`.
 - We use the remainder operator (%) in conjunction with **rand** as follows
`rand() % 6`
 - to produce integers in the range **0 to 5**.

5.10 Random Number Generation (Cont.)

- This is called **scaling**.
- The number 6 is called the **scaling factor**.
- We then **shift** the range of numbers produced by adding 1 to our previous result.
- The output of Fig. 5.7 confirms that the results are in the range 1 to 6—the actual random values chosen might vary by compiler.

```

1 // Fig. 5.11: fig05_11.c
2 // Shifted, scaled random integers produced by 1 + rand() % 6.
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(void)
7 {
8     // loop 20 times
9     for (unsigned int i = 1; i <= 20; ++i) {
10
11         // pick random number from 1 to 6 and output it
12         printf("%10d", 1 + (rand() % 6));
13
14         // if counter is divisible by 5, begin new line of output
15         if (i % 5 == 0) {
16             puts("");
17         }
18     }
19 }

```

Diagram illustrating the components of the random number generation formula $1 + (\text{rand()} \% 6)$:

- shift**: Points to the constant `1` in the formula.
- scaling factor**: Points to the modulus `6` in the formula.

6	6	5	5	6
5	1	1	5	3
6	6	2	4	2
6	2	3	4	1

Fig. 5.11 | Shifted, scaled random integers produced by $1 + \text{rand()} \% 6$.

5.10 Random Number Generation (Cont.)

Rolling a Six-Sided Die 6,000,000 Times

- To show that these numbers occur approximately with *equal likelihood*, let's simulate 6,000,000 rolls of a die with the program of Fig. 5.12.
 - Each integer from 1 to 6 should appear approximately 1,000,000 times.
 - As the program output shows, by **scaling** and **shifting** we've used the rand function to realistically **simulate** the rolling of a six-sided die.

```

1 // Fig. 5.12: fig05_12.c
2 // Rolling a six-sided die 60,000,000 times.
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(void)
7 {
8     unsigned int frequency1 = 0; // rolled 1 counter
9     unsigned int frequency2 = 0; // rolled 2 counter
10    unsigned int frequency3 = 0; // rolled 3 counter
11    unsigned int frequency4 = 0; // rolled 4 counter
12    unsigned int frequency5 = 0; // rolled 5 counter
13    unsigned int frequency6 = 0; // rolled 6 counter
14
15    // loop 60000000 times and summarize results
16    for (unsigned int roll = 1; roll <= 60000000; ++roll) {
17        int face = 1 + rand() % 6; // random number from 1 to 6
18
19        // determine face value and increment appropriate counter
20        switch (face) {
21
22            case 1: // rolled 1
23                ++frequency1;
24                break;
25
26            case 2: // rolled 2
27                ++frequency2;
28                break;
29
30            case 3: // rolled 3
31                ++frequency3;
32                break;
33
34            case 4: // rolled 4
35                ++frequency4;
36                break;
37
38            case 5: // rolled 5
39                ++frequency5;
40                break;
41
42            case 6: // rolled 6
43                ++frequency6;
44                break; // optional
45        }
46    }
47
48    // display results in tabular format
49    printf("%s%13s\n", "Face", "Frequency");
50    printf("    1%13u\n", frequency1);
51    printf("    2%13u\n", frequency2);
52    printf("    3%13u\n", frequency3);
53    printf("    4%13u\n", frequency4);
54    printf("    5%13u\n", frequency5);
55    printf("    6%13u\n", frequency6);
56 }

```

Face	Frequency
1	9999294
2	10002929
3	9995360
4	10000409
5	10005206
6	9996802

- Note the use of the %s conversion specifier to print the **character strings** "Face" and "Frequency" as column headers.

5.10 Random Number Generation (Cont.)

- When *debugging* a program, this **repeatability** is essential for proving that corrections to a program work properly.
 - Function **rand** actually generates **pseudorandom numbers**.
 - Calling **rand** repeatedly produces a sequence of numbers that appears to be random.
 - However, the sequence repeats itself each time the program is executed.
- Once a program has been thoroughly debugged, it can be conditioned to produce a different sequence of random numbers for each execution.

5.10 Random Number Generation (Cont.)

- This is called **randomizing** and is accomplished with the standard library function `srand`.
- Function `srand` takes an unsigned integer argument and **seeds** function `rand` to produce a different sequence of random numbers for each execution of the program.

5.10 Random Number Generation (Cont.)

```
1 // Fig. 5.13: fig05_13.c
2 // Randomizing the die-rolling program.
3 #include <stdlib.h>
4 #include <stdio.h>
5
6 int main(void)
7 {
8     unsigned int seed; // number used to seed the random number generator
9
10    printf("%s", "Enter seed: ");
11    scanf("%u", &seed); // note %u for unsigned int
12
13    srand(seed); // seed the random number generator
14
15    // loop 10 times
16    for (unsigned int i = 1; i <= 10; ++i) {
17
18        // pick a random number from 1 to 6 and output it
19        printf("%10d", 1 + (rand() % 6));
20
21        // if counter is divisible by 5, begin a new line of output
22        if (i % 5 == 0) {
23            puts("");
24        }
25    }
26 }
```

Enter seed: 67				
6	1	4	6	2
1	6	1	6	4
Enter seed: 867				
2	4	6	1	6
1	1	3	6	2
Enter seed: 67				
6	1	4	6	2
1	6	1	6	4

- Function **srand** takes an **unsigned int** value as an argument.
- The conversion specifier **%u** is used to read an **unsigned int** value with **scanf**.
- The function prototype for **srand** is found in **<stdlib.h>**.

5.10 Random Number Generation (Cont.)

- Let's run the program several times and observe the results.
 - Notice that a **different** sequence of random numbers is obtained each time the program is run, provided that a **different seed** is supplied.
- To randomize **without** entering a seed each time, use a statement like

```
srand(time(NULL));
```

 - This causes the computer to read its **clock** to obtain the value for the **seed automatically**.
 - Function **time** returns the **number of seconds** that have passed since midnight on January 1, 1970.
 - This value is converted to an **unsigned integer** and used as the **seed** to the random number generator.
 - The function prototype for **time** is in **<time.h>**.

5.10 Random Number Generation (Cont.)

Generalized Scaling and Shifting of Random Numbers

- The values produced directly by **rand** are always in the range:

$$0 \leq \text{rand}() \leq \text{RAND_MAX}$$

- The following statement simulates rolling a **six-sided** die:

```
face = 1 + rand() % 6;
```

- This statement always assigns an **integer** value (at random) to the variable **face** in the range $1 \leq \text{face} \leq 6$.
- The **width** of this range (i.e., the number of consecutive integers in the range) is 6 and the **starting number** in the range is 1.
- The width of the range is determined by the number used to **scale rand** with the **remainder operator** (i.e., 6), and the **starting number** of the range is equal to the number (i.e., 1) that's added to **rand % 6**.

5.10 Random Number Generation (Cont.)

- We can generalize this result as follows

$$n = a + \text{rand}() \% b;$$

- where a is the **shifting value** (which is equal to the *first* number in the desired range of consecutive integers) and b is the *scaling factor* (which is equal to the *width* of the desired range of consecutive integers).

Program Demo

- Fig. 5.11
- Fig. 5.12
- Fig. 5.13

5.11 Example: A Game of Chance; Introducing enum

- One of the most popular games of chance is a dice game known as “**craps**.” The rules of the game are simple.
 - A player **rolls two dice**.
 - Each die has **six** faces.
 - These faces contain 1, 2, 3, 4, 5, and 6 spots.
 - After the dice have come to rest, the **sum** of the spots on the two upward faces is **calculated**.
 - If the **sum** is 7 or 11 on the first throw, the player **wins**.
 - If the **sum** is 2, 3, or 12 on the first throw (called “craps”), the player **loses** (i.e., the “house” wins).
 - If the sum is 4, 5, 6, 8, 9, or 10 on the first throw, then that sum becomes the player’s “**point**.”
 - » To **win**, you must **continue** rolling the dice **until** you “**make your point**.”
 - » The player **loses** by rolling a 7 **before** making the point.


```

1 // Fig. 5.14: fig05_14.c
2 // Simulating the game of craps
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <time.h> // contains prototype for function time
6
7 // enumeration constants represent game status
8 enum Status { CONTINUE, WON, LOST };
9
10 int rollDice(void); // function prototype
11
12 int main(void)
13 {
14     // randomize random number generator using current time
15     srand(time(NULL));
16
17     int myPoint; // player must make this point to win
18     enum Status gameStatus; // can contain CONTINUE, WON, or LOST
19     int sum = rollDice(); // first roll of the dice
20
21     // determine game status based on sum of dice
22     switch(sum) {
23
24         // win on first roll
25         case 7: // 7 is a winner
26         case 11: // 11 is a winner
27             gameStatus = WON;
28             break;
29
30         // lose on first roll
31         case 2: // 2 is a loser
32         case 3: // 3 is a loser
33         case 12: // 12 is a loser
34             gameStatus = LOST;
35             break;
36
37         // remember point
38         default:
39             gameStatus = CONTINUE; // player should keep rolling
40             myPoint = sum; // remember the point
41             printf("Point is %d\n", myPoint);
42             break; // optional
43     }
44

```

An enumeration, introduced by the keyword **enum**, is a set of integer constants represented by identifiers.

defined to be of a new type—**enum Status**—stores the current status

Values in an **enum** start with **0** and are incremented by **1**.

```

45 // while game not complete
46 while (CONTINUE == gameStatus) { // player should keep rolling
47     sum = rollDice(); // roll dice again
48
49     // determine game status
50     if (sum == myPoint) { // win by making point
51         gameStatus = WON;
52     }
53     else {
54         if (7 == sum) { // lose by rolling 7
55             gameStatus = LOST;
56         }
57     }
58 }
59
60 // display won or lost message
61 if (WON == gameStatus) { // did player win?
62     puts("Player wins");
63 }
64 else { // player lost
65     puts("Player loses");
66 }
67 }
68
69 // roll dice, calculate sum and display results
70 int rollDice(void)
71 {
72     int die1 = 1 + (rand() % 6); // pick random die1 value
73     int die2 = 1 + (rand() % 6); // pick random die2 value
74
75     // display results of this roll
76     printf("Player rolled %d + %d = %d\n", die1, die2, die1 + die2);
77     return die1 + die2; // return sum of dice
78 }

```

Player wins on the first roll

Player rolled 5 + 6 = 11
Player wins

Player wins on a subsequent roll

Player rolled 4 + 1 = 5
Point is 5
Player rolled 6 + 2 = 8
Player rolled 2 + 1 = 3
Player rolled 3 + 2 = 5
Player wins

Player loses on the first roll

Player rolled 1 + 1 = 2
Player loses

Player loses on a subsequent roll

Player rolled 6 + 4 = 10
Point is 10
Player rolled 3 + 4 = 7
Player loses

Program Demo

- Fig. 5.14

5.12 Storage Classes

- In Chapters 2–4, we used **identifiers** for variable names.
 - The **attributes** of variables include **name**, **type**, **size** and **value**.
 - Uses **identifiers** as names for **user-defined functions**.
- Actually, each **identifier** in a program has **other** attributes, including **storage class**, **storage duration**, **scope** and **linkage**.
 - C provides the **storage class specifiers**: **auto**, **register**, **extern** and **static**.

5.12 Storage Classes (Cont.)

- An identifier's **storage class** determines its **storage duration**, **scope** and **linkage**.
 - An identifier's **storage duration** is the **period** during which the identifier **exists in memory**.
 - Some exist **briefly**, some are **repeatedly created and destroyed**, and others exist for the program's **entire execution**.
 - An identifier's **scope** is where the identifier **can be referenced** in a program.
 - Some can be referenced **throughout** a program, others from only **portions** of a program.
 - An identifier's **linkage** determines for a multiple-source-file program whether the identifier is known **only** in the **current** source file or in **any** source file with **proper declarations**.
- This section discusses **storage classes** and **storage duration**.
- Section 5.13 discusses **scope**.
- Chapter 14 discusses identifier **linkage** and programming with **multiple** source files.

5.12 Storage Classes (Cont.)

- The **storage-class** specifiers can be split into

1. automatic storage duration

- Keyword **auto** is used to declare variables of automatic storage duration.
- Variables with automatic storage duration
 - are **created** when the block in which they're defined is entered;
 - they **exist** while the block is active
 - they're **destroyed** when the block is exited.

Local Variables

- Only variables can have automatic storage duration.
- A function's **local** variables (those declared in the parameter list or function body) normally have automatic storage duration.
 - Keyword **auto** explicitly declares variables of automatic storage duration.
 - Local variables have automatic storage duration by **default**, so keyword **auto** is rarely used.
- For the remainder of the text, we'll refer to **variables with automatic storage duration** simply as **automatic variables**.

2. static storage duration

5.12 Storage Classes (Cont.)

Static Storage Class

- Keywords **extern** and **static** are used in the declarations of **identifiers** for variables and functions of **static storage duration**.
 - **Identifiers** of **static storage duration** exist from the time at which the program **begins** execution until the program **terminates**.
 - For **static variables**, storage is allocated and initialized **only once, before** the program begins execution.
 - For **functions**, the name of the function **exists** when the program **begins** execution.
 - Several types of identifiers of **static storage duration** :
 - **external identifiers** (such as **global variables** and **function names**)
 - **local variables** declared with the storage-class specifier **static**

5.12 Storage Classes (Cont.)

- **Global variables** and **function names** are of storage class **extern** by **default**.
- Global variables
 - are created by placing variable declarations *outside* any function definition
 - they retain their values **throughout** the execution of the program
- Global variables and functions can be referenced by any function that follows their **declarations** or **definitions** in the file.
 - This is one reason for using **function prototypes**
 - when we include **stdio.h** in a program that calls **printf**, the function prototype is placed at the start of our file to make the name **printf** known to the rest of the file.

5.12 Storage Classes (Cont.)

- Local variables declared with the keyword **static**
 - are still **known** only in the **function** in which they're defined
 - but **unlike** automatic variables
 - **static** local variables **retain** their value when the function is exited.
- The next time the function is called, the **static** local variable **contains the value** it had when the function last **exited**.
- The following statement declares local variable **count** to be **static** and initializes it to 1.
 - **static int count = 1;**

5.12 Storage Classes (Cont.)

- All numeric variables of **static storage duration** are initialized to **zero** by default if you do not explicitly initialize them.
- Keywords `extern` and `static` have special **meaning** when explicitly applied to **external identifiers**.
- In Chapter 14 we discuss the explicit use of `extern` and `static` with **external** identifiers and **multiple-source-file** programs.

5.13 Scope Rules

- The **scope of an identifier** is the portion of the program in which the identifier can be referenced.
 - When we define a **local variable** in a **block**
 - it can be referenced only following its definition in that **block** or in **blocks nested** within that block.
- The four identifier scopes are **function scope**, **file scope**, **block scope**, and **function-prototype scope**.

5.13 Scope Rules (Cont.)

- **Labels** (identifiers followed by a colon such as **start:**) are the only identifiers with **function scope**.
- **Labels** can be used **anywhere** in the function in which they appear, but **cannot be referenced** outside the function body.
- **Labels** are used in **switch** statements (as **case** labels) and in **goto** statements (see Chapter 14).
- Labels are **hidden** in the function in which they're defined.
 - This hiding—more formally called **information hiding**—is a means of implementing the **principle of least privilege**—a fundamental principle of good software engineering.

5.13 Scope Rules (Cont.)

- An identifier declared outside any function has **file scope**.
 - Such an **identifier** is “known” (i.e., accessible) in all functions from the point at which the identifier is declared **until** the end of the file.
- **Global variables, function definitions, and function prototypes** placed outside a function **all** have file scope.
- **Identifiers** defined inside a block have **block scope**.
 - Block scope **ends** at the terminating right brace (**}**) of the block.
 - **Local** variables defined at the **beginning** of a function have block scope as do function parameters
 - which are considered **local** variables by the function
 - Any block may contain **variable** definitions.

5.13 Scope Rules (Cont.)

- When blocks are **nested**
 - an identifier in an **outer** block has the **same** name as an identifier in an **inner** block
 - the identifier in the outer block is “**hidden**” until the inner block terminates.
- This means that while executing in the inner block
 - the inner block **sees** the value of its own local identifier
 - **not** the value of the identically named identifier in the enclosing block.
- Local variables declared **static** still have block scope
 - even though they exist from **before** program startup.

5.13 Scope Rules (Cont.)

- Thus, *storage duration* does **not** affect the **scope of an identifier**.
 - The only identifiers with **function-prototype scope** are those used in the parameter list of a function prototype.
- **function prototypes do not require names** in the parameter list
 - only **types** are required.
 - If a name is used in the parameter list of a **function prototypes**, the compiler **ignores the name**.
- **Identifiers** used in a **function prototypes** can be **reused** elsewhere in the program without ambiguity.
- Figure 5.16 demonstrates **scoping** issues with **global** variables, **automatic local** variables, and **static local** variables.

```

1 // Fig. 5.16: fig05_16.c
2 // Scoping.
3 #include <stdio.h>
4
5 void useLocal(void); // function prototype
6 void useStaticLocal(void); // function prototype
7 void useGlobal(void); // function prototype
8
9 int x = 1; // global variable
10
11 int main(void)
12 {
13     int x = 5; // local variable to main
14
15     printf("local x in outer scope of main is %d\n", x);
16
17     { // start new scope
18         int x = 7; // local variable to new scope
19
20         printf("local x in inner scope of main is %d\n", x);
21     } // end new scope
22
23     printf("local x in outer scope of main is %d\n", x);
24
25     useLocal(); // useLocal has automatic local x
26     useStaticLocal(); // useStaticLocal has static local x
27     useGlobal(); // useGlobal uses global x
28     useLocal(); // useLocal reinitializes automatic local x
29     useStaticLocal(); // static local x retains its prior value
30     useGlobal(); // global x also retains its value
31
32     printf("\nlocal x in main is %d\n", x);
33 }
34
35 // useLocal reinitializes local variable x during each call
36 void useLocal(void)
37 {
38     int x = 25; // initialized each time useLocal is called
39
40     printf("\nlocal x in useLocal is %d after entering useLocal\n", x);
41     ++x;
42     printf("local x in useLocal is %d before exiting useLocal\n", x);
43 }
44
45 // useStaticLocal initializes static local variable x only the first time
46 // the function is called; value of x is saved between calls to this
47 // function
48 void useStaticLocal(void)
49 {
50     // initialized once
51     static int x = 50;
52
53     printf("\nlocal static x is %d on entering useStaticLocal\n", x);
54     ++x;
55     printf("local static x is %d on exiting useStaticLocal\n", x);
56 }
57
58 // function useGlobal modifies global variable x during each call
59 void useGlobal(void)
60 {
61     printf("\nglobal x is %d on entering useGlobal\n", x);
62     x *= 10;
63     printf("global x is %d on exiting useGlobal\n", x);
64 }

```

This global variable is **hidden** in any block (or function) in which a variable named x is defined.

The variable x with value 7 is automatically destroyed when the block is exited, and the local variable x in the outer block of main is printed again to show that it's no longer hidden.

local x in outer scope of main is 5
 local x in inner scope of main is 7
 local x in outer scope of main is 5

local x in useLocal is 25 after entering useLocal
 local x in useLocal is 26 before exiting useLocal

local static x is 50 on entering useStaticLocal
 local static x is 51 on exiting useStaticLocal

global x is 1 on entering useGlobal
 global x is 10 on exiting useGlobal

local x in useLocal is 25 after entering useLocal
 local x in useLocal is 26 before exiting useLocal

local static x is 51 on entering useStaticLocal
 local static x is 52 on exiting useStaticLocal

global x is 10 on entering useGlobal
 global x is 100 on exiting useGlobal

local x in main is 5

Local variables declared as **static** **retain** their values even when they're out of scope. In the **next** call to this function, static local variable x will contain the value 51.

Program Demo

- Fig. 5.16

5.14 Recursion

- The programs we've discussed are generally structured as **functions** that call one another in a disciplined, **hierarchical** manner.
- For some types of problems, it's useful to have functions **call themselves**.
- A **recursive function** is a function
 - **calls itself** either **directly** or **indirectly** through another function.
- **Recursion** is a complex topic discussed at length in upper-level computer science courses.
- In this section and the next, **simple examples** of recursion are presented.

5.14 Recursion (Cont.)

- Recursive problem-solving approaches have a number of elements in common.
 - A recursive function is called to **solve** a problem.
 - The function actually knows **how** to solve only the **simplest** case(s), or so-called **base case(s)**
 - If the function is called with a **base case**, the function simply **returns** a result.
 - If the function is called with a **more complex** problem, the function **divides** the problem into **two** conceptual pieces:
 - a piece that the function **knows** how to do
 - a piece that it does **not know** how to do.
 - To make recursion feasible, the latter piece must **resemble** the original problem, but be a slightly simpler or smaller version.

5.14 Recursion (Cont.)

- Because this **new** problem looks **like** the **original** problem, the function launches (calls) a fresh copy of itself to go to work on the smaller problem
 - this is referred to as a **recursive call** or the **recursion step**.
 - The **recursion step** also includes the keyword **return**, because its result will be **combined** with the portion of the problem the function knew how to solve to form a result that will be **passed back** to the original caller.

5.14 Recursion (Cont.)

Recursively Calculating Factorials

- The factorial of a nonnegative integer n , written $n!$ (pronounced “ n factorial”), is the product
 - $n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 1$with $1!$ equal to 1, and $0!$ defined to be 1.
- For example, $5!$ is the product $5 * 4 * 3 * 2 * 1$, which is equal to 120.
- The factorial of an integer, number, greater than or equal to 0 can be calculated **iteratively (nonrecursively)** using a `for` statement as follows:

```
factorial = 1;
for (counter = number; counter >= 1; counter--)
    factorial *= counter;
```

5.14 Recursion (Cont.)

- A ***recursive*** definition of the factorial function is arrived at by observing the following relationship:

$$n! = n \cdot (n - 1)!$$

- For example, **5!** is clearly equal to **5 * 4!** as is shown by the following:

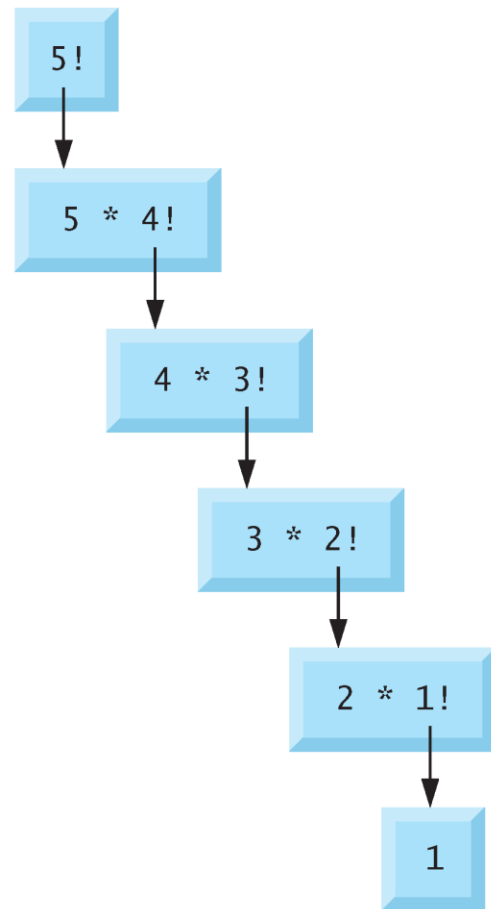
$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$$

$$5! = 5 \cdot (4 \cdot 3 \cdot 2 \cdot 1)$$

$$\mathbf{5! = 5 \cdot (4!)}$$

- The evaluation of 5! would proceed as shown in Fig. 5.17.

a) Sequence of recursive calls



b) Values returned from each recursive call

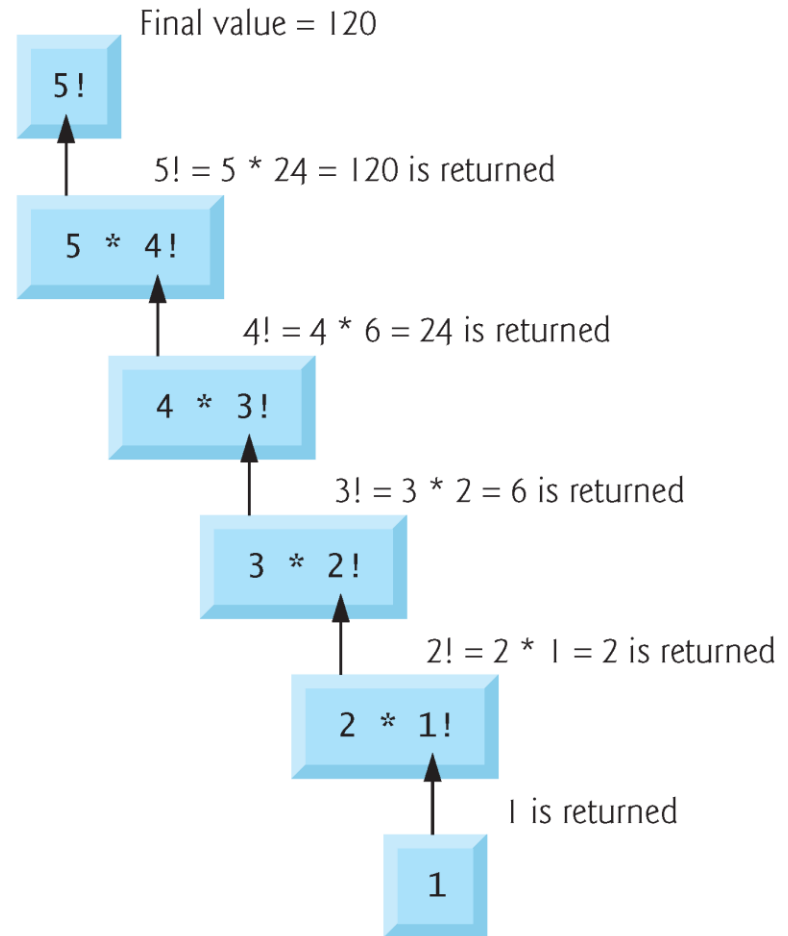


Fig. 5.17 | Recursive evaluation of 5!.

```
1 // Fig. 5.18: fig05_18.c
2 // Recursive factorial function.
3 #include <stdio.h>
4
5 unsigned long long int factorial(unsigned int number);
6
7 int main(void)
8 {
9     // during each iteration, calculate
10    // factorial(i) and display result
11    for (unsigned int i = 0; i <= 21; ++i) {
12        printf("%u! = %llu\n", i, factorial(i));
13    }
14 }
15
16 // recursive definition of function factorial
17 unsigned long long int factorial(unsigned int number)
18 {
19     // base case
20     if (number <= 1) {
21         return 1;
22     }
23     else { // recursive step
24         return (number * factorial(number - 1));
25     }
26 }
```

The C standard specifies that a variable of type **long int** is stored in at least **4 bytes**, and thus may hold a value as large as **+2147483647**.

The conversion specifier **%llu** is used to print **unsigned long long int** values.

If number is indeed **less** than or **equal** to 1, **factorial** returns 1, **no** further recursion is necessary, and the program **terminates**.

n	n!
0!	= 1
1!	= 1
2!	= 2
3!	= 6
4!	= 24
5!	= 120
6!	= 720
7!	= 5040
8!	= 40320
9!	= 362880
10!	= 3628800
11!	= 39916800
12!	= 479001600
13!	= 6227020800
14!	= 87178291200
15!	= 1307674368000
16!	= 20922789888000
17!	= 355687428096000
18!	= 6402373705728000
19!	= 121645100408832000
20!	= 2432902008176640000
21!	= 14197454024290336768

- If number is **greater** than 1, the statement **return number * factorial(number - 1);** expresses the problem as the **product** of **number** and a **recursive** call to **factorial** evaluating the factorial of **number - 1**.
- The call **factorial(number - 1)** is a slightly **simpler** problem than the original calculation **factorial(number)**.

5.14 Recursion (Cont.)

- This points to a **weakness** in C (and most other **procedural** programming languages)
 - namely that the language is **not easily extended** to handle the unique requirements of various applications.
- As we'll see later in the book, C++ is an *extensible* language that,
 - through “**classes**,” allows us to **create new data types**,
 - including ones that could hold **arbitrarily large integers** if we wish.

Program Demo

- Fig. 5.18

5.15 Example Using Recursion: Fibonacci Series

- The Fibonacci series
 - 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

begins with 0 and 1 and has the property that each subsequent Fibonacci number is the **sum of the previous two Fibonacci numbers**.
- The Fibonacci series may be defined recursively as follows:
$$\begin{aligned}\text{fibonacci}(0) &= 0 \\ \text{fibonacci}(1) &= 1 \\ \text{fibonacci}(n) &= \text{fibonacci}(n - 1) + \text{fibonacci}(n - 2)\end{aligned}$$

```

1 // Fig. 5.19: fig05_19.c
2 // Recursive fibonacci function
3 #include <stdio.h>
4
5 unsigned long long int fibonacci(unsigned int n); // function prototype
6
7 int main(void)
8 {
9     unsigned int number; // number input by user
10
11     // obtain integer from user
12     printf("%s", "Enter an integer: ");
13     scanf("%u", &number);
14
15     // calculate fibonacci value for number input by user
16     unsigned long long int result = fibonacci(number);
17
18     // display result
19     printf("Fibonacci(%u) = %llu\n", number, result);
20 }
21

```

- The call to `fibonacci` from `main` is **not** a recursive call, but all subsequent calls to `fibonacci` are **recursive**
- Each time `fibonacci` is invoked, it immediately tests for the **base** case
 - **n is equal to 0 or 1**
 - If this is true, **n** is returned.
 - If **n** is greater than 1, the **recursion** step generates **two** recursive calls, each a slightly **simpler** problem than the original call to `fibonacci`.

Enter an integer: 3
Fibonacci(3) = 2

Enter an integer: 10
Fibonacci(10) = 55

Enter an integer: 20
Fibonacci(20) = 6765

Enter an integer: 30
Fibonacci(30) = 832040

Enter an integer: 40
Fibonacci(40) = 102334155

$\text{fibonacci}(0) = 0$
 $\text{fibonacci}(1) = 1$
 $\text{fibonacci}(n) = \text{fibonacci}(n - 1) + \text{fibonacci}(n - 2)$

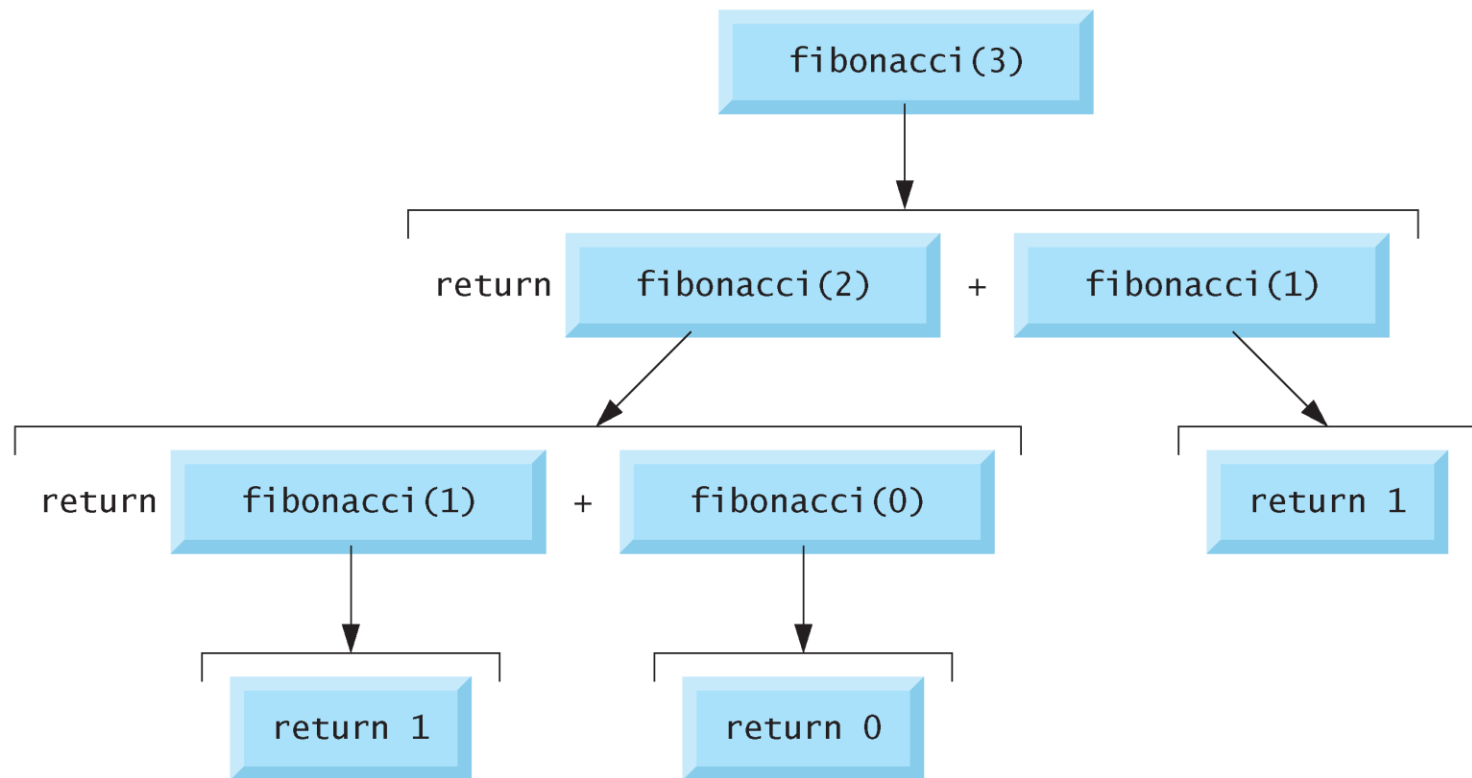


Fig. 5.20 | Set of recursive calls for fibonacci (3).

$\text{fibonacci}(0) = 0$
 $\text{fibonacci}(1) = 1$
 $\text{fibonacci}(n) = \text{fibonacci}(n - 1) + \text{fibonacci}(n - 2)$

Program Demo

- Fig. 5.19

5.16 Recursion vs. Iteration

- Both **iteration** and **recursion** are based on a **control** statement:
 - **Iteration** uses a **repetition** statement
 - **recursion** uses a *selection* statement.
- Both iteration and recursion involve **repetition**:
 - Iteration **explicitly** uses a **repetition** statement
 - recursion achieves **repetition** through *repeated function calls*.
- Iteration and recursion each involve a *termination test*:
 - Iteration terminates when the *loop-continuation condition fails*
 - recursion when a *base case* is *recognized*.

5.16 Recursion vs. Iteration (Cont.)

- Iteration with **counter-controlled** repetition and recursion each *gradually approach termination*:
 - Iteration keeps **modifying** a counter until the counter assumes a value that makes the *loop-continuation condition fail*
 - recursion keeps **producing** simpler versions of the original problem until the *base case is reached*
- Both iteration and recursion can occur *infinitely*:
 - An *infinite loop* occurs with iteration if the loop-continuation test **never** becomes *false*;
 - *infinite recursion* occurs if the recursion step *does not reduce* the problem each time in a manner that converges on the base case.
 - Infinite iteration and recursion typically occur as a result of **errors in a program's logic**.

5.16 Recursion vs. Iteration (Cont.)

- Recursion has many **negatives**.
 - It *repeatedly* invokes the mechanism
 - Consequently the *overhead of function calls*.
 - This can be expensive in both **processor time** and **memory space**.

5.16 Recursion vs. Iteration (Cont.)

- Each recursive call causes *another copy* of the function (actually only the function's variables) to be created;
 - this can **consume** *considerable memory*.
- Iteration normally occurs within a function, so the **overhead** of repeated function calls and extra **memory** assignment is **omitted**.
- So why choose recursion?

5.16 Recursion vs. Iteration (Cont.)

- Figure 5.21 summarizes by chapter the 31 recursion examples and exercises in the text.

Summarizes by chapter the **31** recursion examples and exercises in the text.

Recursion examples and exercises

Chapter 5

Factorial function

Fibonacci function

Greatest common divisor

Multiply two integers

Raising an integer to an integer power

Towers of Hanoi

Recursive main

Visualizing recursion

Chapter 6

Sum the elements of an array

Print an array

Print an array backward

Print a string backward

Check whether a string is a palindrome

Minimum value in an array

Linear search

Binary search

Eight Queens

Chapter 7

Maze traversal

Chapter 8

Printing a string input at the keyboard backward

Chapter 12

Search a linked list

Print a linked list backward

Binary tree insert

Preorder traversal of a binary tree

Inorder traversal of a binary tree

Postorder traversal of a binary tree

Printing trees

Appendix D

Selection sort

Quicksort

Appendix E

Fibonacci function

Fig. 5.21 | Recursion examples and exercises in the text.

Program Exercise 1

- 模擬今彩539 開獎

Program Exercises 2

5.39 (*Recursive Greatest Common Divisor*) The greatest common divisor of integers x and y is the largest integer that evenly divides both x and y . Write a recursive function `gcd` that returns the greatest common divisor of x and y . The `gcd` of x and y is defined recursively as follows: If y is equal to 0, then `gcd`(x , y) is x ; otherwise `gcd`(x , y) is `gcd`(y , $x \% y$), where `%` is the remainder operator.

```
Enter two integers: 32 16
Greatest common divisor of 32 and 16 is 16
```

(*Greatest Common Divisor*) The *greatest common divisor (GCD)* of two integers is the largest integer that evenly divides each of the two numbers. Write function `gcd` that returns the greatest common divisor of two integers.

```
Enter two integers: 16 32
The greatest common divisor of 16 and 32 is 16

Enter two integers: 32 48
The greatest common divisor of 32 and 48 is 16

Enter two integers: 17 51
The greatest common divisor of 17 and 51 is 17

Enter two integers: 51 17
The greatest common divisor of 51 and 17 is 17

Enter two integers: 48 32
The greatest common divisor of 48 and 32 is 16
```


Program Exercise 3

5.31 (*Coin Tossing*) Write a program that simulates coin tossing. For each toss of the coin the program should print `Heads` or `Tails`. Let the program toss the coin 100 times, and count the number of times each side of the coin appears. Print the results. The program should call a separate function `flip` that takes no arguments and returns 0 for tails and 1 for heads. [*Note: If the program realistically simulates the coin tossing, then each side of the coin should appear approximately half the time for a total of approximately 50 heads and 50 tails.*]

```
Heads Tails Tails Heads Heads Heads Tails Heads Heads Heads
Heads Tails Tails Heads Tails Tails Tails Tails Heads Heads
Heads Tails Tails Tails Tails Tails Heads Heads Heads Heads
Tails Heads Heads Tails Heads Tails Heads Tails Tails Tails
Heads Heads Heads Heads Heads Heads Heads Heads Tails Heads
Heads Tails Tails Tails Heads Tails Heads Heads Tails Heads
Heads Heads Tails Heads Tails Tails Heads Heads Heads Tails
Heads Tails Tails Tails Tails Tails Heads Heads Heads Heads
Heads Tails Heads Tails Tails Heads Heads Heads Heads Tails
Tails Tails Heads Tails Tails Tails Tails Heads Heads Heads

The total number of Heads was 55
The total number of Tails was 45
```


Program Exercises 4

5.29 (*Lowest Common Multiple*) The *lowest common multiple (LCM)* of two integers is the smallest positive integer that is a multiple of both numbers. Write a function `lcm` that returns the lowest common multiple of two numbers.

```
Enter two integers: 32 48
The lowest common multiple of 32 and 48 is 96

Enter two integers: 48 32
The lowest common multiple of 48 and 32 is 96

Enter two integers: 17 51
The lowest common multiple of 17 and 51 is 51

Enter two integers: 51 17
The lowest common multiple of 51 and 17 is 51

Enter two integers: 7 91
The lowest common multiple of 7 and 91 is 91
```